

Lazy loading

- Angular lazy loading is a technique used to optimize application performance by loading modules and their associated resources (components, services, etc.) only when they are needed.
- Rather than loading everything at the initial application startup.
- This contrasts with eager loading, where all modules are loaded upfront.
-

Benefits of Lazy Loading:

- **Faster Initial Load Times:** Reduces the size of the initial JavaScript bundle, leading to quicker application startup.
- **Improved Performance:** Reduces the amount of memory and processing power required at the beginning, enhancing overall responsiveness.
- **Efficient Resource Usage:** Modules are only loaded when their corresponding routes are accessed, saving bandwidth and resources for users who may not visit certain parts of the application.
- **Better User Experience:** Users experience less waiting time, especially in large applications with many features.

How Lazy Loading Works:

- **Module Creation:** Create separate feature modules for different logical sections of your application.
- **Routing Configuration:** Configure your application's routing to specify which modules should be lazy-loaded using the `loadChildren` property in the route configuration.
- **Dynamic Loading:** When a user navigates to a route associated with a lazy-loaded module, Angular dynamically fetches and loads that module and its components.

In standalone angular 19

- In Angular 19, lazy loading is most commonly implemented using standalone components and the router's `loadComponent` property.
- This method dynamically imports and loads a component only when the user navigates to its specific route, significantly improving initial application load times.
- A design pattern that loads NgModules as needed.
- Lazy loading helps keep initial bundle sizes smaller, which in turn helps decrease load times.
- This significantly improves the initial load time and overall performance, especially in larger applications.

Example

- Create separate components for product-related pages.

ng generate component products/product-list

ng generate component products/product-detail

- Configure lazy loading in app.routes.ts

Open src/app/app.routes.ts and set up routes.

The key is to use the `loadComponent` property, which takes a dynamic `import()` function pointing to your standalone component.

```
export const routes: Routes = [  
  { path: '', redirectTo: 'products', pathMatch: 'full' },  
  {  
    path: 'products',  
    loadComponent: () => import('./products/product-list/product-list.component')  
      .then(m => m.ProductListComponent),  
  },  
  {  
    path: 'products/:id', // Route for individual product details  
    loadComponent: () => import('./products/product-details/product-details.component')  
      .then(m => m.ProductDetailsComponent),  
  },  
  // ... other routes  
];
```

In app.component.html

```
<nav>
```

```
  <a routerLink="">Home</a> |
```

```
  <a routerLink="/products">Products</a>
```

```
</nav>
```

<http://localhost:420>

main bundle downloaded, but no product-list or product-detail component chunks

navigates to /products/123 Angular dynamically imports and loads the ProductDetailsComponent and its dependencies.

- `loadComponent`: This property in the route configuration is key for lazy loading standalone components. It takes a function that returns a promise, which resolves with the component to be loaded.
- `import('./path/to/component')`: This dynamic import syntax tells Angular to create a separate bundle for the ProductDetailsComponent and load it only when this specific route is activated.
- `.then(m => m.ProductDetailsComponent)`: